Implementation and Evaluation of Joint Optimization of Resource Utilization and Makespan in Cloud Computing using a Genetic Algorithm in CloudSim

Sanskar Shekhar (BTECH/10150/23) and Aman Nautiyal (BTECH/10171/23)
Department of Computer Science and Engineering
Birla Institute of Technology, Mesra, Ranchi
Course: Cloud Computing Lab (IT348) Semester: Seventh

Abstract

Task scheduling in cloud data centres must balance two goals: finishing a workload quickly (low makespan) and keeping virtual machines (VMs) busy (high resource utilization, i.e. low resource wastage). This report reproduces, in CloudSim 5.0, the core problem studied by Dehury, Kumar and Kumar [1]: assigning independent tasks to heterogeneous VMs with a genetic algorithm (GA) that jointly optimizes makespan and utilization. We formulate the problem, implement the GA on the five-VM configuration assigned in Question 8, and compare it with Round Robin, MCT, Min-Min and Max-Min over 50 to 1000 tasks with 10 paired runs per size. The GA lowers mean makespan by 54.5–57.5% relative to Round Robin and raises utilization from 57.4–61.1% to about 99.5%. In paired tests it is significantly better than MCT up to 200 tasks and than Min-Min up to 500 tasks, but it never beats Max-Min: the two tie up to 200 tasks and Max-Min is better by 0.16–0.21% at 500 and 1000 tasks, while running 128 to 5 250 times faster. Further experiments show that (i) the two objectives are almost perfectly aligned in this setting, so the fitness weights barely matter; (ii) the GA's shortfall at 1000 tasks comes from a mutation rate that is too high for long chromosomes, and a rate of $1/n$ closes it; (iii) the paper's own GA, re-implemented exactly from its Algorithm 1, is within about 1% of Max-Min up to 200 tasks but 9–29% worse at 500 and 1000 tasks, because its mutation rate of 0.1 and its 100-iteration budget do not scale to long chromosomes; and (iv) modelling each VM as one processing element is accurate for large workloads, whereas giving each VM one processing element per core lengthens small-workload makespans by up to 39%. The closed-form model and the CloudSim execution agree to within 0.46% on average (1.37% at worst).

Index Terms

Cloud computing, CloudSim, task scheduling, genetic algorithm, makespan, resource utilization, load balancing.

I. Introduction

CLOUD data centres execute large numbers of user tasks on shared pools of virtualized resources. How the tasks are mapped to VMs affects both user-facing performance and provider-side efficiency.

A. Background

Scheduling independent tasks on heterogeneous VMs is NP-hard, so heuristic and metaheuristic methods are widely used. Genetic algorithms [4] search the large space of task-to-VM assignments with selection, crossover and mutation and need no problem-specific gradient. Classical list-scheduling heuristics such as Min-Min and Max-Min [5], [6] give strong baselines at very low cost.

B. Motivation

A provider wants VMs to be well used (little idle, wasted capacity); a user wants the workload to finish early. A scheduler that optimizes both together is attractive to both, which is the motivation of [1].

C. Problem Being Addressed

Given a batch of independent tasks and a set of heterogeneous VMs, find a task-to-VM mapping that jointly minimizes makespan and maximizes resource utilization, following the GA-based approach of Dehury *et al.* [1], and evaluate it in CloudSim [2].

II. Problem Statement and Objectives

Problem statement (Question 8). Design and implement a CloudSim-based simulation that reproduces the core problem addressed in [1], implement the scheduling approach based on its methodology, design and justify an experimental configuration, evaluate the paper's objectives quantitatively on the VM configuration of Table I, validate the implementation against the paper, and analyse its behaviour, strengths, limitations and scalability.

Objectives.

1) Formulate the problem: cloud resources, tasks, VMs, decision variables, objectives and constraints (Section III).
2) Translate the formulation into a CloudSim simulation (Sections IV–VI).
3) Implement a GA scheduler that jointly optimizes makespan and utilization (Section IV-C).
4) Compare it with Round Robin, Minimum Completion Time (MCT), Min-Min and Max-Min, with paired statistics (Section VIII).
5) Justify every configuration choice, by argument or by experiment (Sections V and IX).
6) Validate the implementation against the paper and justify each difference (Section X).
7) Analyse behaviour, strengths, limitations and scalability (Section XI).

III. The Reference Paper and Problem Formulation

A. Summary of the Reference Paper

Dehury, Kumar and Kumar [1] address load balancing in the cloud by scheduling independent tasks on VMs with a genetic algorithm. They treat it as a multi-objective problem with two objectives, *makespan* and *resource wastage*. Resource utilization is the summed task completion time divided by the makespan times the number of VMs, and wastage is one minus utilization. The two objectives are combined by a weighted sum, $f = w_1 \cdot \text{Makespan} + w_2 \cdot \text{ResW}$, whose weights are left to the application, subject to each task running on one VM and to the CPU and memory capacity of every VM. A solution is an integer vector whose i-th gene is the VM of task i. The GA (Algorithm 1 of [1]) uses roulette-wheel selection with probability $P_i = (F_{\max} - F_i)/\sum_p (F_{\max} - F_p)$, one-point crossover that turns each pair of parents into two children, per-gene mutation that moves a task to another random VM with probability m_r, and $(\mu + \lambda)$ elitist replacement that keeps the best P of parents and children. It is implemented in Python 3.10 on Google Colab and evaluated on the Cloud–Fog Computing task dataset (100–500 tasks) with 60 heterogeneous VMs modelled on nine Amazon EC2 I8g instance types, with a population of 100, 100 iterations and $m_r = 0.1$. Compared over 20 runs with Particle Swarm Optimization, Artificial Bee Colony, Black Widow Optimization and random allocation, it reports reductions of up to 16.04% in makespan and 32.52% in resource wastage. Table X (Section X) compares each element of [1] with our implementation.

B. Problem Elements

The six elements that Question 8 asks us to identify are:

- **Cloud resources:** one data centre with two physical hosts that run the five VMs (Section V).
- **Tasks/workload:** a batch of n independent, non-preemptive tasks $T = \{t_1, \ldots, t_n\}$ with lengths L_i in million instructions (MI), all available at time 0.
- **Virtual machines:** $V = \{v_1, \ldots, v_m\}$, $m = 5$, with processing capacity C_j (MIPS) derived from the core counts in Table I.

- **Decision variables:** the binary assignment x_{ij} of task t_i to VM v_j (Eq. 2).
- **Objectives:** minimize makespan M and maximize average utilization $\bar{U}$, equivalently minimize resource wastage $1 - \bar{U}$ (Eqs. 4–6).
- **Constraints:** each task runs on exactly one VM (Eq. 3) and each VM runs its tasks one at a time. In [1] the CPU and memory demands of the tasks placed on a VM must also stay within its capacity; our tasks have no memory demand and each needs one processing element, which every VM provides, so these capacity constraints are always satisfied here.

C. Mathematical Formulation

The expected execution time of task t_i on VM v_j is

$$E_{ij} = \frac{L_i}{C_j}. \tag{1}$$

The decision variable is

$$x_{ij} = \begin{cases} 1 & \text{if task } t_i \text{ is assigned to VM } v_j, \\ 0 & \text{otherwise,} \end{cases} \tag{2}$$

subject to

$$\sum_{j=1}^{m} x_{ij} = 1, \quad \forall i \in \{1, \ldots, n\}. \tag{3}$$

The busy time (load) of VM v_j and the makespan are

$$B_j = \sum_{i=1}^{n} x_{ij} E_{ij}, \qquad M = \max_{1 \le j \le m} B_j. \tag{4}$$

The average resource utilization, the resource wastage and the degree of imbalance are

$$\bar{U} = \frac{1}{m} \sum_{j=1}^{m} \frac{B_j}{M} = \frac{B_{\text{avg}}}{M}, \qquad W = 1 - \bar{U}, \qquad DI = \frac{B_{\max} - B_{\min}}{B_{\text{avg}}}, \tag{5}$$

where B_{avg}, $B_{\max}$ and $B_{\min}$ are the mean, largest and smallest VM loads. W is the fraction of VM time, up to the makespan, that is left idle. These are exactly the utilization and resource-wastage definitions of [1] (its Eqs. 6 and 7). These definitions reproduce the utilization and imbalance values in the result files: for the 100-task Round Robin run of Fig. 7 the per-VM loads give $\bar{U} = 0.667$ and $DI = 1.057$.

The GA minimizes one fitness value that combines the two objectives:

$$F(x) = w_1 \frac{M}{M_{\text{LB}}} + w_2 \left(1 - \bar{U}\right), \qquad M_{\text{LB}} = \frac{\sum_{i=1}^{n} L_i}{\sum_{j=1}^{m} C_j}, \qquad w_1 = w_2 = 0.5, \tag{6}$$

where M_{LB} is the makespan of a perfectly balanced, perfectly divisible schedule (total work divided by total capacity). It normalizes the makespan term so that both terms are dimensionless. Because $M \ge M_{\text{LB}}$ and $\bar{U} \le 1$, $F \ge w_1 = 0.5$. This is a lower bound, not necessarily an attainable value, because tasks cannot be split. The paper [1] uses the same weighted sum with the raw makespan, $f = w_1 M + w_2(1 - \bar{U})$, and does not fix the weights. In its experiments makespans are of the order of 0.15 (Table 13 of [1]), the same scale as the wastage term. Here makespans range from 24 s to over 1000 s, so a raw sum would make the wastage term negligible; dividing by M_{LB} keeps both terms on the same scale, which preserves the intent of [1] and keeps the weights meaningful.

Remark on the two objectives. With these definitions the objectives are closely aligned. A perfectly balanced schedule ($B_j = M$ for all j) minimizes both. They can diverge only when work is moved to a slower VM, which raises B_{avg} (and hence $\bar{U}$) without finishing earlier. Section IX-A measures how much this matters in practice.

IV. System Architecture and Methodology

A. Architecture

Fig. 1 shows the simulation. A workload generator creates the cloudlets; each scheduler turns them into a task-to-VM mapping; the broker binds every cloudlet to its VM and CloudSim executes the mapping on the simulated hosts. The same mapping is also evaluated with the closed-form model of Section III, which allows the CloudSim execution to be verified (Section X). Metrics are written to CSV files and processed in Python.

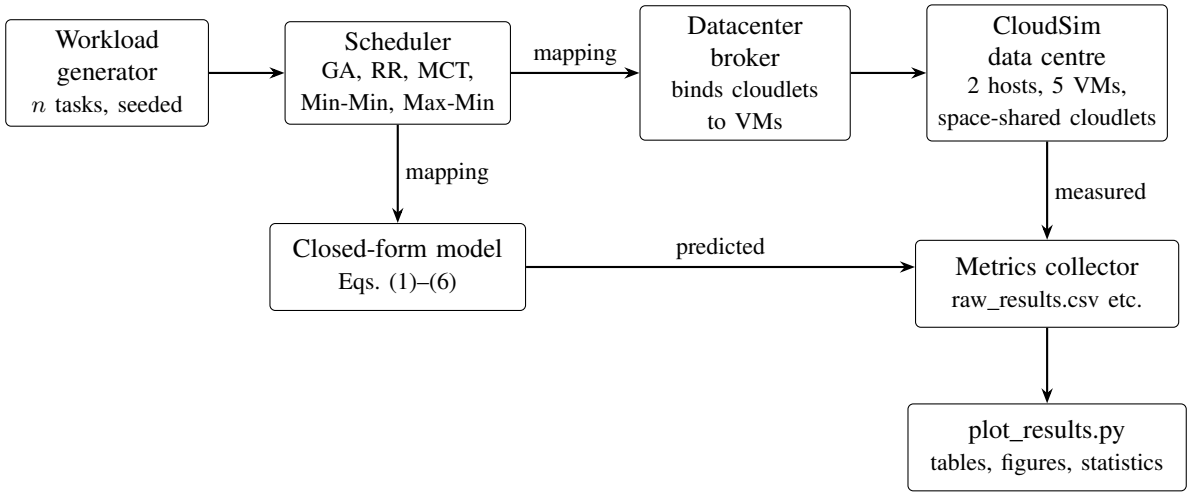


Fig. 1. Architecture of the CloudSim-based simulation.

B. Baseline Schedulers

- **Round Robin (RR):** assigns task i to VM $i \bmod m$, ignoring VM speed.
- **MCT:** takes tasks in arrival order and gives each to the VM on which it would complete earliest.
- **Min-Min / Max-Min [5], [6]:** repeatedly compute every unscheduled task's earliest completion time; Min-Min schedules the task with the smallest such time first, Max-Min the task with the largest.

C. Genetic Algorithm

A chromosome is an integer vector of length n whose i-th gene is the index of the VM assigned to task t_i, so every chromosome satisfies Eq. (3) by construction. Algorithm 1 gives the complete procedure; the parameters are justified in Table III.

Each generation therefore contains the two elite chromosomes and $P - 2$ new offspring. Two-point crossover copies parent 1 and replaces the genes between two random positions with those of parent 2. For the comparison of Section IX-C, the GA of [1] is also implemented exactly as given in its Algorithm 1 (class `PaperGA`), and our GA can seed its initial population with the Max-Min schedule.

D. Workflow

(1) Create the CloudSim data centre, hosts and VMs; (2) generate the task set; (3) run each scheduler to obtain a mapping and time it; (4) bind the cloudlets to the VMs and run the simulation; (5) record the measured makespan, utilization and imbalance together with the closed-form predictions; (6) repeat for every task count and run; (7) compute tables, figures and paired statistics.

V. Experimental Configuration

A. VM Configuration

Table I lists the five VMs given in Question 8.

Algorithm 1 GA-based task scheduling

Create a population of P chromosomes with every gene drawn uniformly from $\{1, \ldots, m\}$
for $g = 1$ **to** G **do**
  Evaluate the fitness F (Eq. 6) of every chromosome; update the best chromosome found so far
  $next \leftarrow$ copies of the $E = 2$ fittest chromosomes ▷ elitism
  while $|next| < P$ **do**
    Select parents p_1, p_2, each by a tournament of $k = 3$ random chromosomes
    With probability p_c: $child \leftarrow$ two-point crossover of p_1, p_2; otherwise $child \leftarrow p_1$
    For each gene, with probability p_m, reassign the task to a uniformly random VM
    Append $child$ to $next$
  $population \leftarrow next$
return the best chromosome found as the task-to-VM mapping

TABLE I
VIRTUAL MACHINE CONFIGURATION (QUESTION 8) AND DERIVED CAPACITY

VM	CPU (cores)	RAM (GB)	Bandwidth (Gbps)	Capacity C_j (MIPS)
VM1	4	8	1.0	4000
VM2	2	4	0.5	2000
VM3	8	16	2.0	8000
VM4	6	8	1.0	6000
VM5	2	8	0.5	2000

B. Assumptions and Their Justification

Question 8 and the summary of [1] do not fix the hosts, workload or several modelling details. Table II lists every assumption we made and why.

C. GA Parameters

Table III gives the GA parameters. The values sit inside the ranges commonly recommended for GAs [9], [10], and Section IX-B tests each one experimentally.

VI. IMPLEMENTATION

The simulation is written in Java against the `org.cloudbus.cloudsim` API of the CloudSim toolkit [2]. The results in this report were produced with CloudSim 5.0, the version used in the lab [3]; the code also compiles against CloudSim 3.0.3. It has five classes:

- `Scheduling`: the closed-form evaluation (loads, makespan, utilization, imbalance, M_{LB}) and the four baseline schedulers.
- `GeneticScheduler`: the GA of Algorithm 1, with an option to seed the population with the Max-Min schedule.
- `PaperGA`: the GA of [1] exactly as given in its Algorithm 1, used for the comparison in Section IX-C.
- `Q8Main`: builds the data centre, hosts and VMs, generates the cloudlets, runs and times every scheduler, executes each mapping in CloudSim, and writes `raw_results.csv` (one row per task count, run and scheduler, with both the predicted and the CloudSim-measured values), `convergence.csv`, `vm_load.csv` and `environment.txt`.
- `Q8Experiments`: the additional studies of Section IX (weight sweep, parameter sensitivity, GA variants and the multi-PE check).

The Python script `plot_results.py` reads the CSV files with pandas [8], computes the paired statistics with SciPy [13] and draws the figures with Matplotlib [7]. It also writes every table of Sections VIII–IX

TABLE II
ASSUMPTIONS AND JUSTIFICATION

Aspect	Choice	Justification
Core speed	1000 MIPS per core; $C_j = \text{cores}_j \times 1000$	Gives the VMs the 4:2:8:6:2 capacity ratio implied by Table I. Only ratios matter: a different per-core speed rescales every time by the same factor and changes no comparison.
VM model	Each VM is *one* processing element (PE) of C_j MIPS	Matches $E_{ij} = L_i/C_j$ of the formulation, which treats a VM's capacity as one pool. Its effect is measured in Section IX-D: it is small for large workloads and large for small ones.
Cloudlet scheduler	Space-shared	Tasks on a VM run one after another, so the VM load is the sum of execution times (Eq. 4).
RAM and bandwidth	Given to each VM as in Table I, but not part of the objective	The tasks are compute-bound and carry no data. In CloudSim's default data centre, RAM and bandwidth only constrain where VMs are placed, and a cloudlet's execution time depends on its length and the PE speed. The 300-byte input/output sizes and full utilization models of the cloudlets therefore do not change any result.
Hosts	2 identical hosts, each 32 cores (4 PEs of 8000 MIPS), 64 GB RAM, 20 Gbps, time-shared VM scheduler	Typical dual-socket server size. Their capacity (64 cores, 128 GB) exceeds the 22 cores and 44 GB the VMs need, so placement never constrains the schedule and the study isolates task scheduling, which is the problem of [1]. A PE of 8000 MIPS lets the fastest VM fit on one PE.
Workload sizes	50, 100, 200, 500, 1000 tasks	From 10 to 200 tasks per VM, covering the regime where scheduling decisions matter most (few tasks per VM) to the regime where greedy heuristics are near-optimal.
Task lengths	Uniform on $[1000, 20000]$ MI (mean about 10 500 MI)	A 20:1 spread between the longest and shortest task, so task order matters to list heuristics; execution times range from 0.125 s to 10 s.
Arrival	All tasks at $t = 0$ (batch)	The formulation is a static mapping of a known set of independent tasks.
Runs and seeds	10 runs per size; task set seed $42 + 1000 \cdot run + n$; the GA's random generator uses the same seed	Every scheduler sees the same task set within a run, so results can be compared pairwise (Section VIII-B); the whole experiment is reproducible bit for bit.

TABLE III
GENETIC ALGORITHM PARAMETERS

Parameter	Value	Justification
Population P	100	Halving it or doubling it changes makespan by at most 0.73 points relative to Max-Min (Table VII); 100 balances quality and run time.
Generations G	300	Up to 500 tasks the best fitness stops improving by generation 242 or earlier (Fig. 8); at 1000 tasks it is still improving at generation 292, and 100 generations are clearly too few.
Initial population	Random, no seeding	A plain GA as in [1]; the seeded hybrid is tested separately (Section IX-C).
Selection	Tournament, $k = 3$	Gives steady selection pressure that does not depend on the scale of F [11]; roulette wheel is compared in Section IX-C.
Crossover	Two-point, $p_c = 0.8$	p_c within the usual 0.6–0.95 range [9], [10]; $p_c = 0.6$ and 1.0 give similar results (Table VII).
Mutation	Per-gene random VM, $p_m = 0.02$	Two expected changes per child at 100 tasks. Section IX-B shows it is too high for 1000-task chromosomes, where $p_m = 1/n$ [12] is better; [1] uses 0.1.
Elitism	Best 2 copied unchanged	Guarantees the best fitness never gets worse; [1] achieves the same with $(\mu + \lambda)$ replacement.
Fitness weights	$w_1 = w_2 = 0.5$	Equal priority to the two objectives; the full sweep $w_1 \in \{0, \ldots, 1\}$ is reported in Section IX-A.

as a LaTeX file, so the tables in this report are generated directly from the result files. The source code and results are available at https://github.com/shekharsanskar9/Project8.

VII. Experimental Setup

- **Hardware:** a virtual machine with 2 vCPUs of an Intel Xeon processor at 2.10 GHz and 8 GB of RAM, running Linux 6.18 (x86-64). The scheduling times below were measured on this machine and scale with its speed.
- **Software:** CloudSim 5.0 with Apache Commons Math 3.4.1; the code was developed in Eclipse IDE on JDK 8 as prescribed by the lab [3], compiled for Java 8 (`-source 8 -target 8`) and run on OpenJDK 21.0.12. Analysis with Python 3.13, pandas 3.0.5, SciPy 1.18.1 and Matplotlib 3.11.2.
- **Workload and algorithms:** 50, 100, 200, 500 and 1000 tasks on the five VMs of Table I; Round Robin, MCT, Min-Min, Max-Min and GA.
- **Runs:** 10 runs per task count and scheduler (250 CloudSim simulations for the main experiment), plus 950 for the additional studies. Results are reported as mean $\pm$ standard deviation.
- **Timing:** before any measurement every scheduler is run on throw-away workloads so that Java's just-in-time compiler has compiled it (warm-up). A GA run is timed directly. The heuristics finish in microseconds, below the timer's useful resolution, so each is called repeatedly for at least 50 ms and the mean time per call is recorded. Tables report the median over the 10 runs.
- **Statistics:** because all schedulers receive the same task set within a run, schedulers are compared on per-run differences: mean difference with a 95% t-confidence interval, the number of runs won, tied and lost, and a two-sided Wilcoxon signed-rank test [14].
- **Metrics:** makespan M (s), makespan relative to the lower bound M/M_{LB}, average utilization $\bar{U}$, degree of imbalance DI, and scheduling (algorithm execution) time (ms).

VIII. Results

A. Main Comparison

Table IV gives the results for every task count. Fig. 2 shows makespan against workload size. Round Robin grows much faster than the other four schedulers, whose curves overlap at this scale. Fig. 3 therefore plots makespan divided by the lower bound M_{LB}, which makes the differences between the four visible.

B. Paired Comparison of the GA with the Strong Baselines

Table V compares the GA with MCT, Min-Min and Max-Min run by run. A negative difference means the GA finished earlier.

C. GA Convergence

Fig. 8 shows the best fitness in the first run of every task count, plotted as its gap above the bound $F \geq 0.5$. In the 100-task run the best fitness falls from 0.872 in generation 1 to below 0.52 by generation 21 and reaches its final value 0.5054 at generation 174, about 1.1% above the bound. Up to 500 tasks the last improvement comes at generation 242 or earlier. At 1000 tasks the curve is still falling, with its last improvement at generation 292 (final fitness 0.5039): the GA has not converged.

D. Console Output

Fig. 9 shows the summary that `Q8Main` prints at the end of the run that produced Table IV. The printed scheduling times are means over the 10 runs; Table IV reports medians.

TABLE IV
PERFORMANCE COMPARISON (MEAN OVER 10 RUNS)

Tasks	Scheduler	Makespan (s)	M/M_{LB}	Util. (%)	Imbalance (DI)	Sched. time (ms)[a]
50	Round Robin	57.36 ± 7.27	2.374	57.4	1.341	0.000159
	MCT	25.35 ± 1.82	1.052	93.5	0.158	0.0004
	Min-Min	26.15 ± 1.85	1.085	89.3	0.325	0.00947
	Max-Min	24.39 ± 1.67	1.012	99.2	0.017	0.00917
	GA	24.39 ± 1.65	1.012	99.5	0.014	48.1
100	Round Robin	106.18 ± 12.09	2.225	61.1	1.231	0.000259
	MCT	49.20 ± 2.79	1.030	96.2	0.083	0.000734
	Min-Min	49.28 ± 3.09	1.032	95.3	0.121	0.0344
	Max-Min	48.20 ± 2.89	1.009	99.5	0.012	0.0351
	GA	48.27 ± 2.93	1.011	99.5	0.012	85
200	Round Robin	226.46 ± 20.33	2.365	58.2	1.322	0.000465
	MCT	97.67 ± 2.98	1.020	97.8	0.044	0.00142
	Min-Min	99.02 ± 3.07	1.035	95.8	0.086	0.188
	Max-Min	96.42 ± 3.03	1.007	99.7	0.006	0.249
	GA	96.53 ± 3.09	1.008	99.5	0.009	159
500	Round Robin	538.82 ± 18.76	2.280	59.9	1.270	0.00139
	MCT	238.72 ± 5.37	1.010	99.2	0.019	0.00377
	Min-Min	239.64 ± 4.97	1.014	98.7	0.036	1.35
	Max-Min	238.01 ± 5.21	1.007	99.7	0.006	1.55
	GA	238.39 ± 5.29	1.009	99.5	0.011	416
1000	Round Robin	1079.32 ± 32.87	2.256	60.2	1.247	0.00284
	MCT	482.57 ± 7.15	1.009	99.4	0.011	0.00752
	Min-Min	482.06 ± 7.49	1.008	99.6	0.010	5.27
	Max-Min	481.34 ± 7.22	1.006	99.7	0.005	6.12
	GA	482.35 ± 7.34	1.008	99.5	0.011	784

[a] Median of the 10 runs, measured after JIT warm-up; for the heuristics, the mean of repeated calls lasting at least 50 ms.

TABLE V
PAIRED MAKESPAN DIFFERENCES (GA MINUS BASELINE, 10 RUNS PER SIZE)

Comparison	Tasks	Mean diff. (s) ± 95% CI	Mean diff. (%)	GA wins/ties/losses	Wilcoxon p
GA − MCT	50	-0.96 ± 0.40	-3.75	10/0/0	0.002
	100	-0.93 ± 0.37	-1.90	10/0/0	0.002
	200	-1.14 ± 0.53	-1.17	9/0/1	0.004
	500	-0.33 ± 0.39	-0.14	6/0/4	0.131
	1000	-0.22 ± 0.86	-0.05	6/0/4	0.625
GA − Min-Min	50	-1.76 ± 0.40	-6.68	10/0/0	0.002
	100	-1.01 ± 0.31	-2.04	10/0/0	0.002
	200	-2.50 ± 0.15	-2.52	10/0/0	0.002
	500	-1.25 ± 0.56	-0.53	10/0/0	0.002
	1000	+0.29 ± 0.58	+0.06	3/0/7	0.322
GA − Max-Min	50	-0.00 ± 0.05	+0.00	5/0/5	1.000
	100	+0.08 ± 0.10	+0.15	4/0/6	0.105
	200	+0.11 ± 0.14	+0.11	4/0/6	0.160
	500	+0.38 ± 0.17	+0.16	1/0/9	0.004
	1000	+1.01 ± 0.57	+0.21	1/0/9	0.004

With 10 pairs the smallest possible two-sided Wilcoxon p is 0.002 (all 10 runs in the same direction).

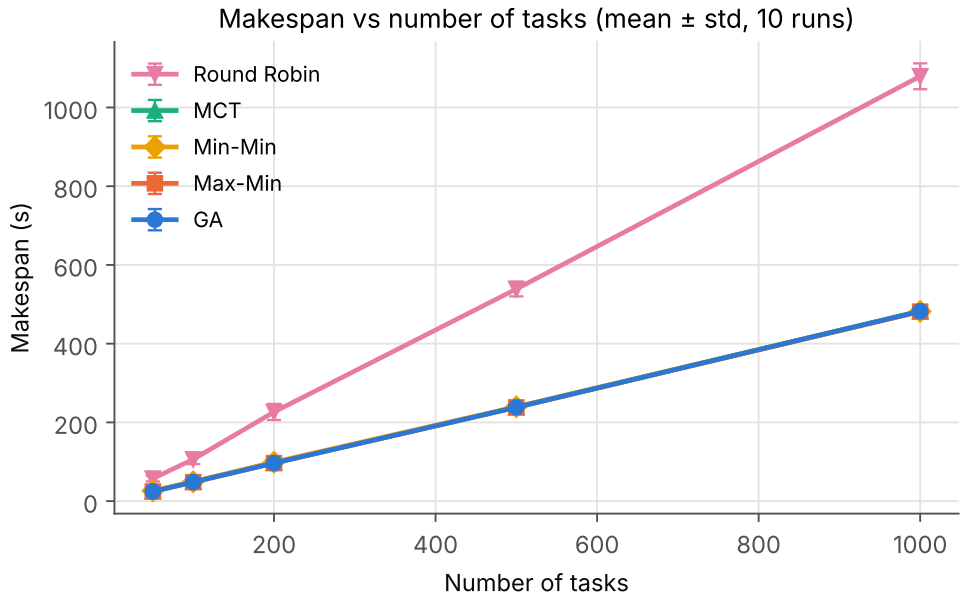


Fig. 2. Makespan versus number of tasks (mean ± standard deviation over 10 runs).

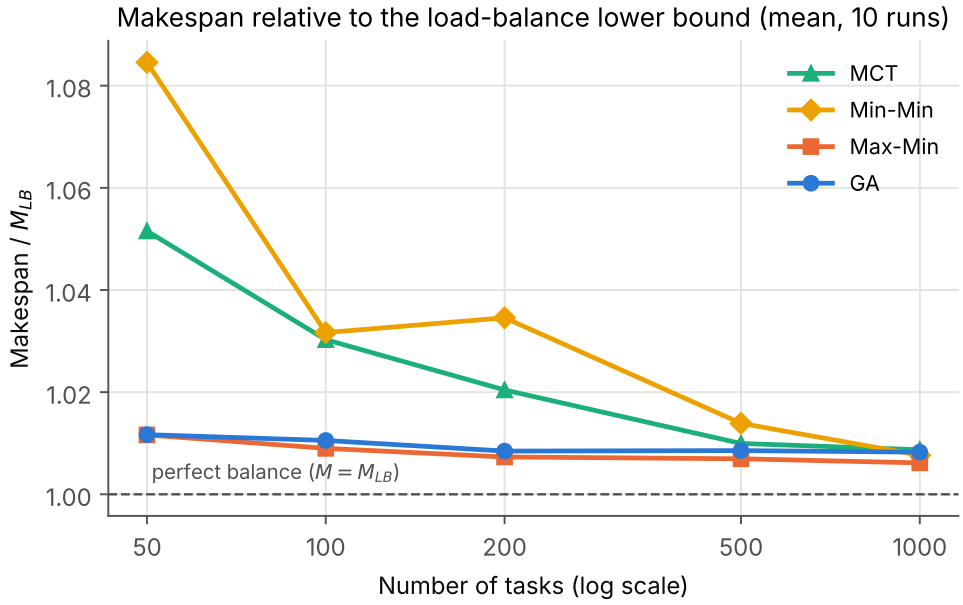


Fig. 3. Makespan divided by the load-balance lower bound M_{LB} (mean over 10 runs; Round Robin, at 2.23–2.37, is omitted). The dashed line is a perfectly balanced schedule.

IX. Additional Experiments

A. *Fitness Weights: Is There a Trade-off?*

To test whether the two objectives actually compete, the GA was run with $w_1 \in \{0, 0.25, 0.5, 0.75, 1\}$ and $w_2 = 1 - w_1$ on the same task sets (100 and 500 tasks, 10 runs each). Table VI and Fig. 10 show the result. The last column is the total VM busy time divided by its minimum possible value $m\,M_{\mathrm{LB}}$; it would rise above 1 if the GA shifted work to slower VMs to raise utilization.

Across the whole sweep the mean makespan changes by less than 0.2% (48.20–48.28 s at 100 tasks, 238.15–238.53 s at 500 tasks) and utilization stays between 99.26% and 99.60%. Total busy time stays

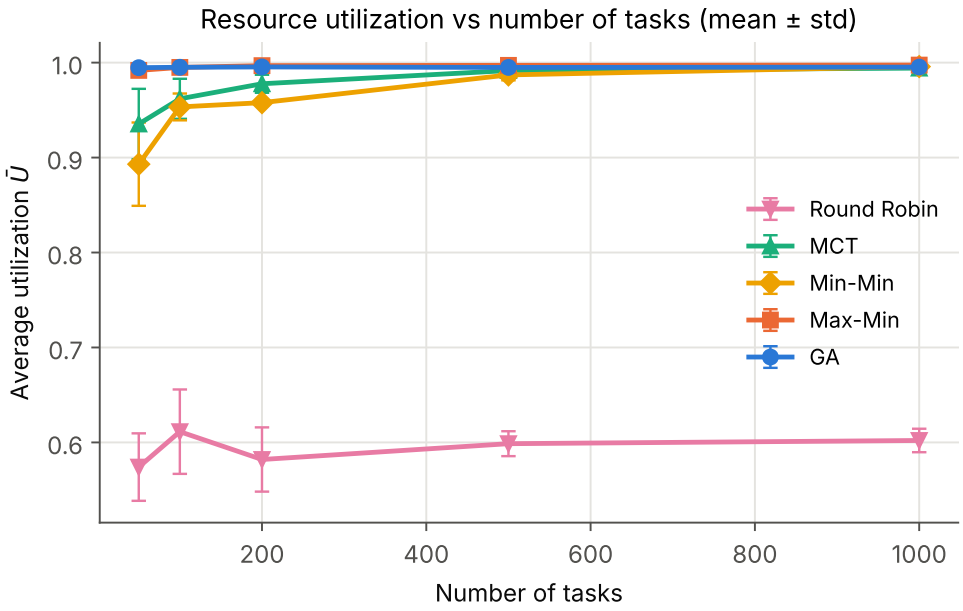


Fig. 4. Average resource utilization versus number of tasks.

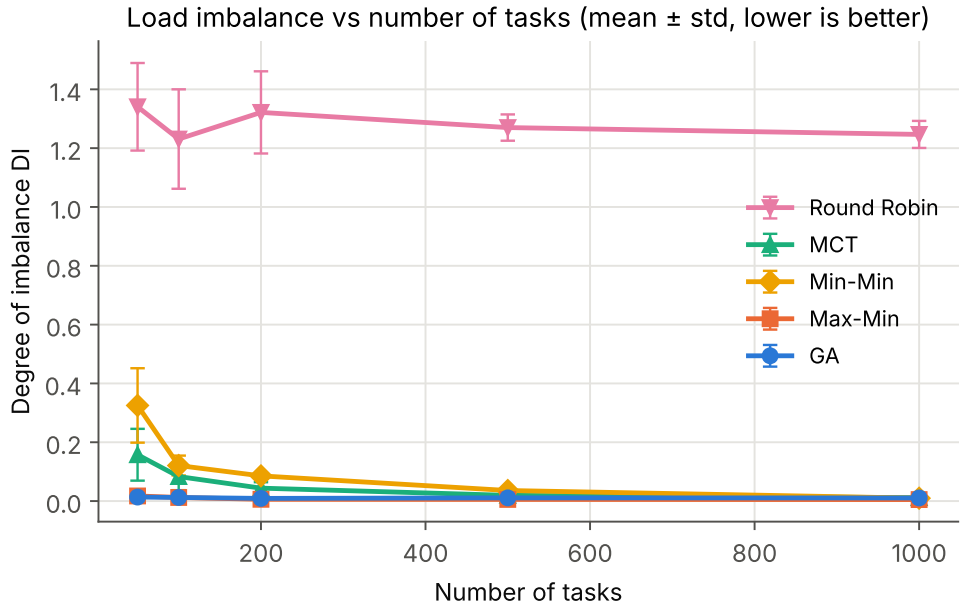


Fig. 5. Degree of imbalance versus number of tasks (lower is better).

within 0.6% of its minimum for every weight, including $w_1 = 0$, where the fitness rewards only utilization: the GA does not raise utilization by shifting work to slower VMs. The highest utilization and lowest makespan come at the same weight ($w_1 = 0.75$ for both sizes), so the objectives do not trade off: in this setting optimizing either one optimizes the other.

B. GA Parameter Sensitivity

Each GA parameter was varied one at a time around the default on the same 10 task sets for 100 and 1000 tasks. Table VII reports M/M_{LB}, the mean makespan difference from Max-Min on the same task

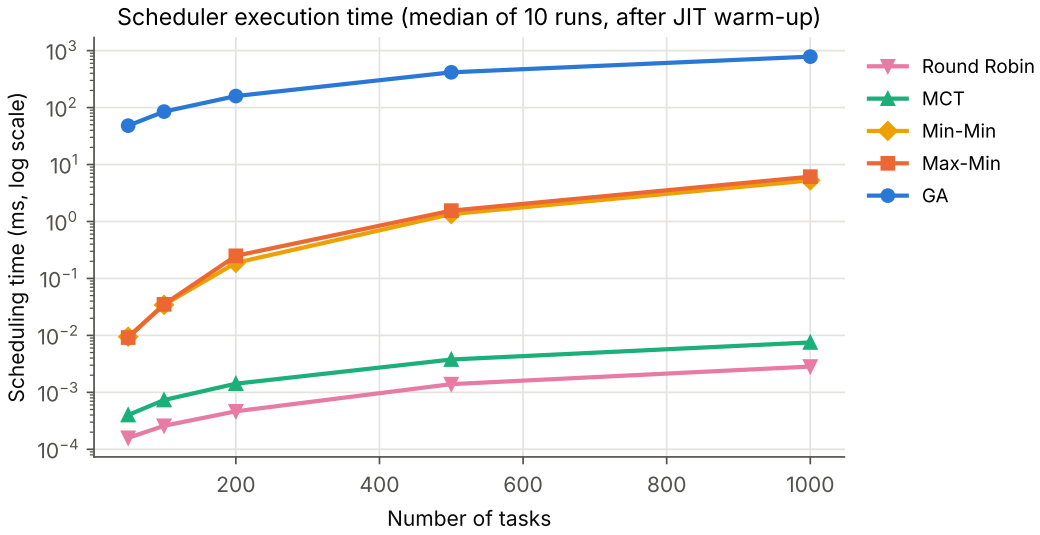


Fig. 6. Scheduling (algorithm execution) time versus number of tasks, log scale.

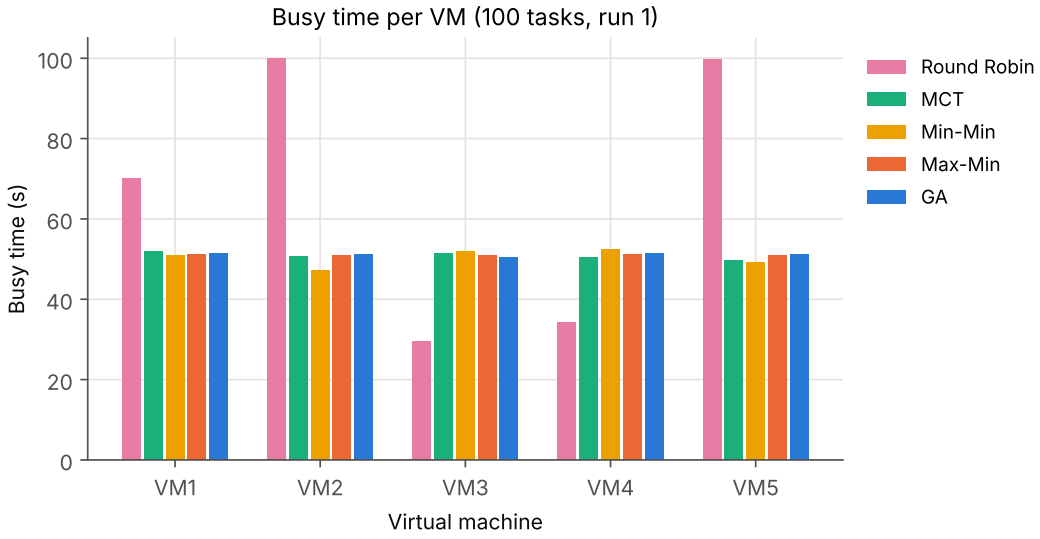


Fig. 7. Busy time per VM for each scheduler (100 tasks, first run).

TABLE VI
EFFECT OF THE FITNESS WEIGHTS (GA, MEAN OVER 10 RUNS)

Tasks	w_1	w_2	Makespan (s)	M/M_{LB}	Util. (%)	DI	Busy time / mM_{LB}
100	0.00	1.00	48.25	1.0102	99.48	0.012	1.0050
	0.25	0.75	48.27	1.0105	99.42	0.013	1.0047
	0.50	0.50	48.27	1.0105	99.51	0.012	1.0056
	0.75	0.25	48.20	1.0091	99.52	0.010	1.0043
	1.00	0.00	48.28	1.0107	99.26	0.019	1.0033
500	0.00	1.00	238.47	1.0089	99.51	0.011	1.0039
	0.25	0.75	238.53	1.0092	99.43	0.014	1.0035
	0.50	0.50	238.39	1.0086	99.49	0.011	1.0035
	0.75	0.25	238.15	1.0075	99.60	0.009	1.0035
	1.00	0.00	238.33	1.0083	99.46	0.015	1.0028

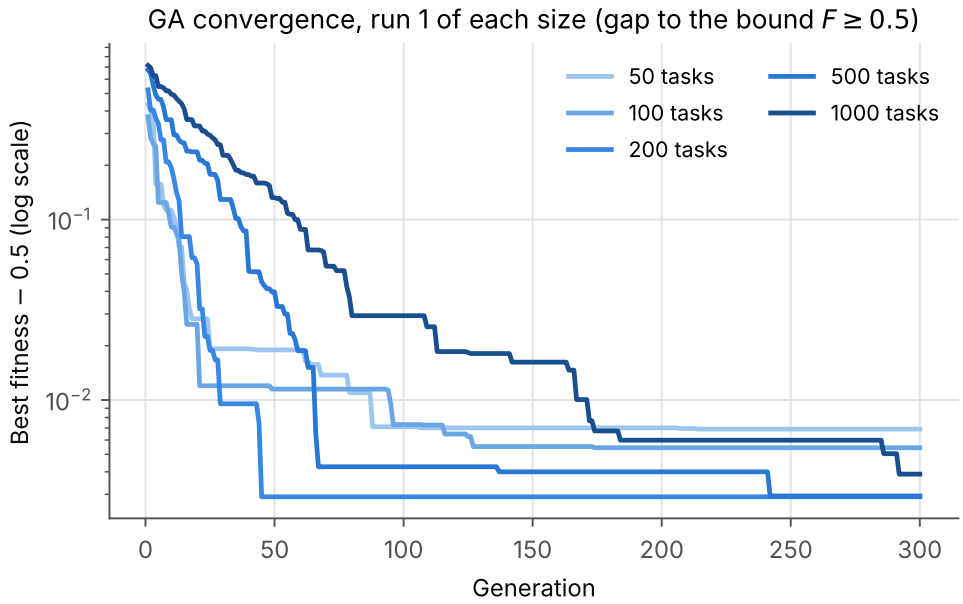


Fig. 8. GA convergence for the first run of each task count; the gap $F - 0.5$ to the lower bound is shown on a log scale (lower is better).

```
finished task count 50
finished task count 100
finished task count 200
finished task count 500
finished task count 1000

tasks  algorithm    makespan(s)       util  imbalance      algo_ms
50     RoundRobin          57.36     0.5740     1.3407         0.00
50     MCT                 25.35     0.9354     0.1575         0.00
50     MinMin              26.15     0.8930     0.3252         0.01
50     MaxMin              24.39     0.9918     0.0174         0.01
50     GA                  24.39     0.9947     0.0136        49.06
100    RoundRobin         106.18     0.6113     1.2310         0.00
100    MCT                 49.20     0.9619     0.0834         0.00
100    MinMin              49.28     0.9533     0.1208         0.04
100    MaxMin              48.20     0.9950     0.0123         0.04
100    GA                  48.27     0.9951     0.0123        85.60
200    RoundRobin         226.46     0.5820     1.3216         0.00
200    MCT                 97.67     0.9777     0.0440         0.00
200    MinMin              99.02     0.9578     0.0857         0.62
200    MaxMin              96.42     0.9968     0.0061         0.25
200    GA                  96.53     0.9954     0.0091       162.69
500    RoundRobin         538.82     0.5987     1.2700         0.00
500    MCT                238.72     0.9916     0.0194         0.00
500    MinMin             239.64     0.9871     0.0359         1.31
500    MaxMin             238.01     0.9971     0.0056         1.65
500    GA                 238.39     0.9949     0.0109       407.23
1000   RoundRobin        1079.32     0.6020     1.2467         0.00
1000   MCT                482.57     0.9942     0.0114         0.01
1000   MinMin             482.06     0.9958     0.0098         5.05
1000   MaxMin             481.34     0.9974     0.0051         6.22
1000   GA                 482.35     0.9953     0.0106       787.15

Validation: max relative difference between formula makespan and CloudSim makespan = 1.3658%
Wrote raw_results.csv, convergence.csv, vm_load.csv, environment.txt
```

Fig. 9. Console output of `Q8Main` for the reported run.

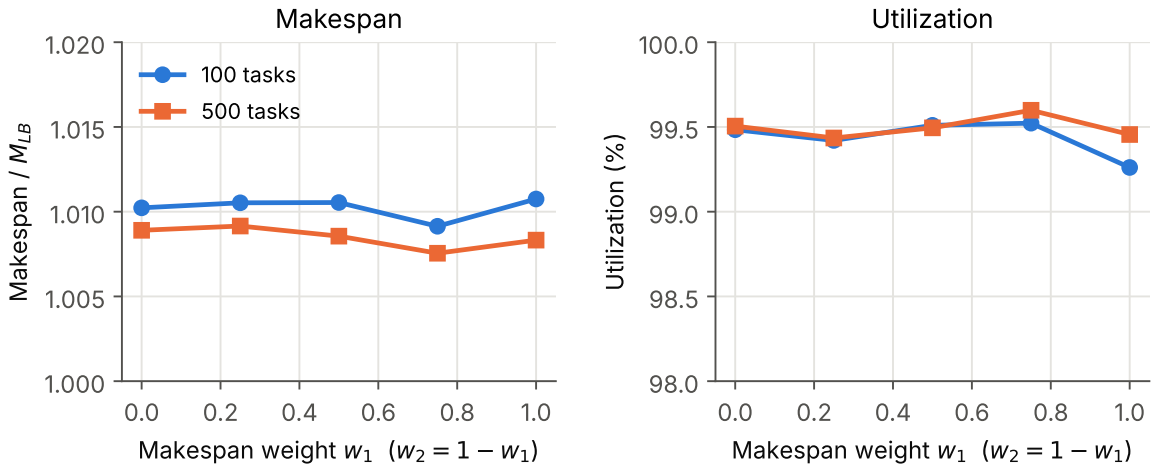


Fig. 10. Makespan (relative to M_{LB}) and utilization of the GA for different fitness weights (mean of 10 runs). The axes span 2% so that the small size of the effect is visible.

set, the number of runs in which the GA beat Max-Min, and the median scheduling time. The default is $P = 100$, $G = 300$, $p_c = 0.8$, $p_m = 0.02$.

TABLE VII
GA PARAMETER SENSITIVITY (ONE FACTOR AT A TIME, 10 RUNS)

	100 tasks				1000 tasks			
Setting	M/M_{LB}	ΔMM (%)	beats	ms	M/M_{LB}	ΔMM (%)	beats	ms
Default	1.0105	+0.15	4/10	85	1.0083	+0.21	1/10	774
$P = 50$	1.0080	-0.10	6/10	41	1.0156	+0.94	1/10	386
$P = 200$	1.0095	+0.05	6/10	168	1.0072	+0.11	2/10	1592
$G = 100$	1.0104	+0.14	3/10	29	1.0737	+6.72	0/10	255
$G = 1000$	1.0084	-0.06	7/10	280	1.0074	+0.13	2/10	2569
$p_c = 0.6$	1.0097	+0.07	6/10	82	1.0094	+0.32	0/10	776
$p_c = 1.0$	1.0091	+0.01	5/10	88	1.0086	+0.24	0/10	790
$p_m = 0.005$	1.0101	+0.11	5/10	83	1.0064	+0.03	5/10	782
$p_m = 1/n$	1.0091	+0.01	5/10	85	1.0060	-0.01	4/10	795
$p_m = 0.05$	1.0131	+0.41	0/10	85	1.1658	+15.87	0/10	789

ΔMM: mean makespan difference from Max-Min on the same task set (negative = GA better); beats: runs in which the GA's makespan is below Max-Min's; ms: median scheduling time.

At 100 tasks no setting moves the GA more than 0.41% from Max-Min, and several settings (population 50, 1000 generations) beat it slightly on average. At 1000 tasks the mutation rate dominates. With $p_m = 0.02$ a 1000-gene child receives about 20 random reassignments; near a balanced schedule almost every such change makes balance worse, so most offspring are worse than their parents. With $p_m = 1/n$ (one expected change per child, the classic recommendation [12]) or $p_m = 0.005$, the GA closes the gap to Max-Min (-0.01% and $+0.03\%$), while $p_m = 0.05$ is 15.87% worse. A budget of 100 generations is far too small at 1000 tasks (+6.72%). The main results keep the default parameters so that all experiments are consistent, but $p_m = 1/n$ is the better choice for large workloads.

C. The Paper's GA and a Seeded Hybrid

Two further GAs were run on all task sets. The first is the GA of [1] implemented exactly as its Algorithm 1 (class `PaperGA`): roulette-wheel selection with $P_i \propto F_{\max} - F_i$, one-point crossover

producing two children per pair, per-gene mutation and $(\mu + \lambda)$ replacement, with the paper's settings of $P = 100$, 100 iterations and $m_r = 0.1$. To separate the effect of the mutation rate it was also run with $m_r = 1/n$. Both use the fitness of Eq. (6), so only the algorithm differs from ours. The second is our GA with one initial chromosome set to the Max-Min schedule. Table VIII and Fig. 11 show the results.

TABLE VIII
GA VARIANTS COMPARED WITH MAX-MIN (MEAN OVER 10 RUNS)

Tasks	Variant	Makespan (s)	M/M_{LB}	Util. (%)	vs Max-Min (%)	W/T/L	ms
50	GA	24.39	1.0117	99.47	+0.005	5/0/5	48
	Paper's GA, its settings	24.66	1.0230	97.79	+1.128	1/0/9	22
	Paper's GA, $m_r = 1/n$	24.46	1.0150	99.15	+0.337	2/0/8	19
	GA, Max-Min seeded	24.40	1.0123	99.27	+0.070	3/0/7	52
100	GA	48.27	1.0105	99.51	+0.154	4/0/6	85
	Paper's GA, its settings	48.70	1.0195	98.25	+1.038	0/0/10	29
	Paper's GA, $m_r = 1/n$	48.26	1.0104	99.53	+0.135	5/0/5	28
	GA, Max-Min seeded	48.20	1.0090	99.58	+0.004	3/1/6	83
200	GA	96.53	1.0085	99.54	+0.115	4/0/6	159
	Paper's GA, its settings	97.36	1.0171	98.54	+0.976	0/0/10	63
	Paper's GA, $m_r = 1/n$	96.55	1.0087	99.63	+0.142	4/0/6	58
	GA, Max-Min seeded	96.43	1.0074	99.68	+0.011	0/8/2	176
500	GA	238.39	1.0086	99.49	+0.160	1/0/9	416
	Paper's GA, its settings	259.98	1.0998	94.37	+9.218	0/0/10	141
	Paper's GA, $m_r = 1/n$	242.39	1.0254	98.66	+1.833	0/0/10	129
	GA, Max-Min seeded	238.01	1.0070	99.71	+0.000	0/10/0	398
1000	GA	482.35	1.0083	99.53	+0.209	1/0/9	784
	Paper's GA, its settings	622.72	1.3017	85.74	+29.371	0/0/10	278
	Paper's GA, $m_r = 1/n$	572.66	1.1968	90.38	+18.951	0/0/10	260
	GA, Max-Min seeded	481.34	1.0062	99.74	+0.000	0/10/0	788

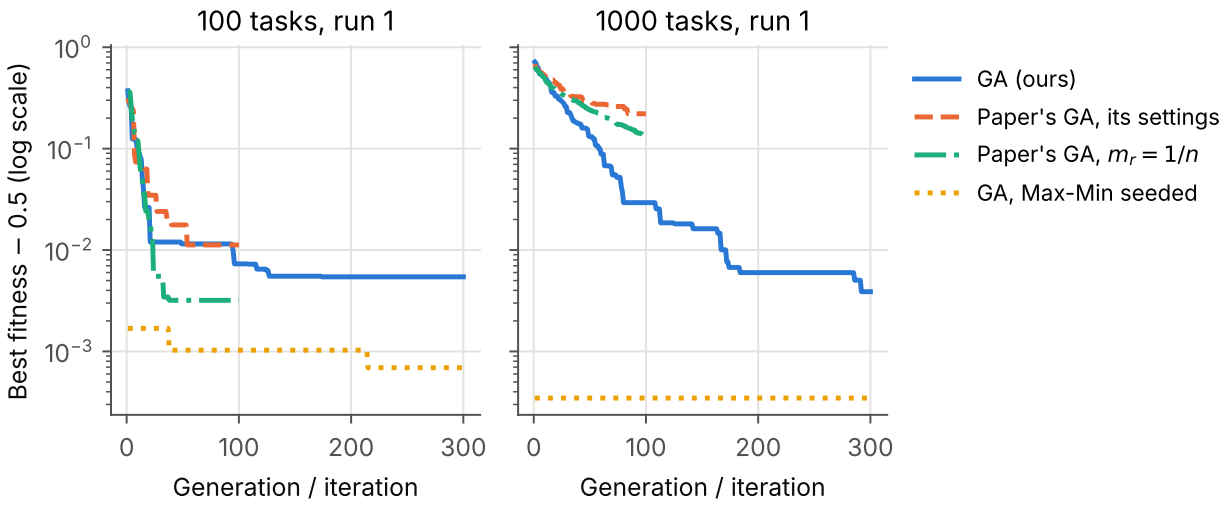


Fig. 11. Convergence of the GA variants in the first run with 100 and 1000 tasks (gap to the bound $F \geq 0.5$, log scale). The paper's GA runs for its 100 iterations.

With its own settings the paper's GA is 0.98–1.13% worse than Max-Min at 50–200 tasks, the range of workloads it was designed for, and falls far behind at 500 and 1000 tasks (9.22% and 29.37% worse, with utilization down to 85.7%). The convergence curves show why. At 1000 tasks a mutation rate of 0.1 changes about 100 genes of every child, so almost every child is worse than its parents and the

$(\mu + \lambda)$ step discards it: the best fitness does not change between iteration 90 and 100 (0.7207). With $m_r = 1/n$ the paper's GA matches ours at 100 and 200 tasks (0.14% from Max-Min) in about a third of the time (28 ms against 85 ms at 100 tasks), and at 1000 tasks it is still improving at iteration 100, so there the 100-iteration budget is the limit (18.95% worse). Its selection scheme is not the cause: probabilities proportional to $F_{\max} - F_i$ give the worst chromosome no chance and put strong pressure on the best. What matters is the mutation rate and the iteration budget, which [1] sets for 100–500 tasks spread over 60 VMs (fewer than 10 tasks per VM), whereas here each VM receives 10 to 200 tasks.

The Max-Min-seeded GA is never worse than Max-Min by more than 0.07% on average, and at 500 and 1000 tasks its result has exactly the fitness and makespan of the Max-Min seed in all 20 runs: within 300 generations the GA could not improve on it. At 50–200 tasks it found schedules with a lower closed-form fitness than Max-Min in 21 of 30 runs; in 15 of these, CloudSim measured a slightly *longer* makespan (by 0.17% on average). Differences this small are below the 0.46% average gap between the closed-form model and CloudSim (Section X-A), so they should not be read as real wins or losses.

D. Effect of Modelling a VM as One Processing Element

To quantify the single-PE assumption of Table II, every mapping of the main experiment was executed a second time with each VM modelled as cores_j PEs of 1000 MIPS (hosts of 32 PEs of 1000 MIPS), so that a c-core VM runs up to c tasks in parallel. Table IX gives the change in CloudSim makespan.

TABLE IX
MAKESPAN CHANGE WHEN VMS HAVE ONE PE PER CORE (%, MEAN OVER 10 RUNS)

Tasks	Round Robin	MCT	Min-Min	Max-Min	GA
50	+5.73	+33.71	+29.53	+37.84	+38.60
100	+2.78	+17.63	+18.69	+18.49	+18.96
200	+1.65	+8.58	+6.17	+8.15	+8.02
500	+0.52	+3.12	+3.09	+3.20	+2.79
1000	+0.27	+1.30	+1.50	+1.57	+1.39

With one PE per core, a task can use only one core's speed: a 20 000 MI task takes 20 s on any VM instead of 2.5 s on VM3. With few tasks per core the last long tasks then finish late, so makespan grows by 29.5–38.6% at 50 tasks for the four load-balancing schedulers. The effect shrinks as the number of tasks per core grows (1.3–1.6% at 1000 tasks). The single-PE model is the one that matches the formulation of [1] and is accurate for large workloads, but at small workloads absolute makespans depend on it. The ranking of the four load-balancing schedulers also changes under the multi-PE model at every size except 100 tasks, while Round Robin stays last everywhere. Differences of a few tenths of a percent between the GA, Max-Min, Min-Min and MCT are therefore model-dependent.

X. VALIDATION

A. Verification of the CloudSim Model Against the Formulation

Every mapping is evaluated twice: with the closed-form model of Section III and by executing it in CloudSim. Over all 250 runs of the main experiment the CloudSim makespan differs from the formula by 0.46% on average and 1.37% at most, and it is the higher of the two in 95% of runs, because the simulator's event processing adds small delays; utilization differs by at most 0.0066. For the 100-task Round Robin run, the per-VM busy times of Fig. 7 also reproduce the reported utilization (0.667) and imbalance (1.057). This confirms that the simulation implements the formulation: independent tasks, each assigned to exactly one VM, VM load equal to the sum of task execution times, and makespan equal to the largest load.

B. Correspondence with the Reference Paper

Table X compares each element of [1] with our implementation and justifies every difference.

TABLE X
CORRESPONDENCE BETWEEN THE PAPER AND THIS IMPLEMENTATION

Element	Paper [1]	This implementation	Same? / justification
Problem	Load balancing: independent tasks on VMs, each task on one VM	Batch of independent tasks on the 5 VMs of Question 8	Same.
Objectives	Makespan and resource wastage $1 - U_{VM}$ (Eqs. 5–7 of [1])	M and $W = 1 - \bar{U}$ (Eqs. 4–5)	Same definitions.
Fitness	$w_1 \cdot$ Makespan $+ w_2 \cdot$ ResW; weights not fixed	$w_1 M/M_{\mathrm{LB}} + w_2 W$, $w_1 = w_2 = 0.5$	Makespan normalized so both terms have the same scale (Section III-C); weights shown not to matter here (Section IX-A).
Constraints	One VM per task; CPU and memory capacity of each VM	One VM per task; capacity always satisfied	Tasks have no memory demand and need one PE each (Table II).
Encoding	Integer vector, gene i = VM of task i	Same	Same.
Initial population	Random	Random	Same.
Selection	Roulette wheel, $P_i \propto F_{\max} - F_i$	Tournament, $k = 3$	Different. The paper's GA is reproduced exactly in Section IX-C; its gap comes from the mutation rate and budget, not from selection.
Crossover	One-point, two children per pair, always applied	Two-point, one child, $p_c = 0.8$	Different; included in the reproduction of Section IX-C.
Mutation	Per-gene move to another random VM, $m_r = 0.1$	Per-gene random VM, $p_m = 0.02$	Same operator. A rate of 0.1 is too high for long chromosomes here (Sections IX-B and IX-C).
Replacement	$(\mu + \lambda)$: best P of parents and children	Generational, best 2 kept	Both elitist: the best fitness never gets worse.
GA parameters	$P = 100$, 100 iterations, $m_r = 0.1$	$P = 100$, $G = 300$, $p_c = 0.8$, $p_m = 0.02$	More generations because our largest workload has 1000 tasks (Table III, Section IX-B).
Environment	Cloud–Fog task dataset (100–500 tasks); 60 VMs of 9 AWS EC2 I8g types, 105 MIPS per vCPU	The 5 VMs of Question 8; seeded synthetic tasks, 50–1000	VMs fixed by Question 8; a seeded synthetic workload keeps the study reproducible (Table II).
Simulator	Own Python 3.10 simulation on Google Colab	CloudSim 5.0 (Java)	Required by Question 8.
Baselines	PSO, ABC, BWO, random allocation	RR, MCT, Min-Min, Max-Min	Classic heuristics, including strong list schedulers; metaheuristic baselines are left for future work.
Runs and statistics	20 independent runs, ANOVA	10 paired runs, Wilcoxon signed-rank test	Paired design: all schedulers see the same task sets.
Metrics	Makespan, resource wastage, fitness, degree of imbalance, task completion time	Makespan, $\bar{U}$ and W, DI, scheduling time	Covers both objectives of [1].

C. Comparison with the Paper's Results

The study in [1] uses different data, 60 VMs and different baselines (Table X), so its numbers cannot be reproduced directly; the comparison is qualitative.

Where the results agree. Like [1], the GA strongly improves on a naive allocation. In [1] it lowers the makespan at 500 tasks from 0.91 under random allocation to 0.69, and resource wastage from about 0.38–0.60 to 0.22–0.40. Here, against Round Robin, it cuts makespan by 54.5–57.5% and wastage W from about 39–43% of VM time to about 0.5%. In both studies imbalance falls as the number of tasks grows (Fig. 5 for MCT and Min-Min; Fig. 12 of [1]), because many small tasks can be spread more evenly than a few large ones.

Where they differ. In [1] the GA is best in fitness and makespan among the metaheuristics compared (ANOVA, $p < 0.0001$), but the paper's own ANOVA finds no significant difference in resource wastage ($p = 0.063$; ABC has the lowest mean). Here, too, the GA has no advantage over the strongest competitor: it ties Max-Min up to 200 tasks and loses by 0.16–0.21% at 500 and 1000 tasks (9 of 10 runs, $p = 0.004$); under the closed-form model Max-Min wins every run from 200 tasks upward. Max-Min is a strong list heuristic that already ends less than 1.2% above the lower bound on this five-VM problem, and [1] does not compare against it. The wastage reached in [1] (0.22–0.41) is much higher than here (about 0.005) because with fewer than 10 tasks per VM a schedule cannot be balanced finely, whereas here every VM receives 10 to 200 tasks. Finally, the paper's GA with its own settings is about 1% behind Max-Min at the workload sizes it was designed for and falls further behind at larger sizes (Section IX-C), so its parameters need to be scaled with the number of tasks.

XI. Analysis and Discussion

A. What Worked

Makespan. The GA lowers mean makespan by 57.5%, 54.5%, 57.4%, 55.8% and 55.3% relative to Round Robin for 50, 100, 200, 500 and 1000 tasks (Table IV). Round Robin ignores VM speed, so the slow two-core VMs receive as many tasks as the eight-core VM. Fig. 7 shows the consequence: for 100 tasks the Round Robin loads range from 29.5 s (VM3) to 100.1 s (VM2), whereas every other scheduler keeps all five VMs between 47.2 s and 52.5 s.

Utilization and imbalance. The GA keeps utilization at 99.5% for every workload size, against 57.4–61.1% for Round Robin (Fig. 4), and its degree of imbalance stays between 0.009 and 0.014, against 1.23–1.34 (Fig. 5).

Against MCT and Min-Min. The GA is significantly faster-finishing than MCT at 50, 100 and 200 tasks (3.75%, 1.90% and 1.17% lower makespan; 10, 10 and 9 of 10 runs) and than Min-Min at 50 to 500 tasks (6.68% down to 0.53%; 10 of 10 runs at every size). At 1000 tasks neither difference is significant.

B. What Did Not Work

Against Max-Min. The GA does not outperform Max-Min at any size. The difference is not significant up to 200 tasks, and Max-Min is better at 500 and 1000 tasks (Table V). Seeding the GA with the Max-Min schedule makes it at least as good as Max-Min but no better at 500 and 1000 tasks (Table VIII).

Convergence at 1000 tasks. With the default $p_m = 0.02$ the GA is still improving at generation 292 of 300 (Fig. 8). Section IX-B shows the cause is the mutation rate, not only the generation budget: $p_m = 1/n$ matches Max-Min within 0.01% at the same cost.

Cost. The GA takes a median of 48 ms for 50 tasks and 784 ms for 1000 tasks, against 0.009 ms and 6.1 ms for Max-Min. It is therefore 128 to 5 250 times slower (Fig. 6). In absolute terms this is small: 784 ms is 0.16% of the 482 s makespan it schedules, and the overhead stays below 0.2% of the makespan at every size. For a batch scheduler this cost is acceptable; it would matter only for very frequent rescheduling or much larger problems.

C. Why the Results Look This Way

With only five VMs and tens to hundreds of tasks per VM, a greedy completion-time heuristic already places work almost perfectly: Max-Min schedules the longest tasks first and fills the gaps with short ones, ending less than 1.2% above the lower bound at every size (Fig. 3). Utilization cannot exceed 100%, so the remaining room for any scheduler is under about 1%. MCT and Min-Min make early placement mistakes that matter when there are few tasks per VM, which is why the GA's advantage over them is largest at 50 tasks and fades as the workload grows. The weight sweep (Section IX-A) shows why the joint objective behaves like a single one here: a balanced schedule maximizes utilization and minimizes makespan at the same time.

D. Trends

All load-balancing schedulers approach the lower bound as the workload grows (M/M_{LB} falls from 1.012–1.085 at 50 tasks to 1.006–1.009 at 1000 tasks); Round Robin stays at 2.23–2.37 times the bound. The heuristics' scheduling time grows roughly quadratically for Min-Min and Max-Min and linearly for MCT and Round Robin, while the GA's grows linearly with the number of tasks (its cost per generation is proportional to $P \cdot n$).

E. Strengths

The GA gives near-perfect balance (utilization about 99.5%, DI about 0.01) at every size, beats MCT and Min-Min on small and medium workloads, and its fitness function can be extended with further objectives (energy, cost, deadlines) without changing the algorithm, which a list heuristic cannot do.

F. Limitations

1) The GA never clearly beats Max-Min, and its advantage over MCT and Min-Min disappears at 1000 tasks.
2) Its scheduling time is two to three orders of magnitude higher than the heuristics', although under 0.2% of the makespan.
3) The experiments use five VMs, independent tasks and one synthetic workload family, so the conclusions may not hold for larger or more heterogeneous clusters, real traces or dependent-task workflows.
4) Absolute makespans at small workloads depend on the single-PE VM model, and the order of the four load-balancing schedulers changes under the multi-PE model (Section IX-D).
5) With 10 runs per setting, differences below about 0.2% are within the run-to-run variation and the 0.46% average model-to-simulator gap, and should not be over-interpreted; the paired tests in Table V mark which differences are significant.
6) Scheduling times were measured on one 2-vCPU machine; the ratios between schedulers are more reliable than the absolute values.

G. Scalability

Makespan grows linearly with the number of tasks for all schedulers: about 0.48 s per task for the GA (and MCT, Min-Min and Max-Min) at every size, against 1.06–1.15 s for Round Robin. The GA's scheduling time grows from 48 ms to 784 ms for a 20-fold increase in tasks (16-fold), but a fixed budget of 300 generations becomes insufficient for long chromosomes unless the mutation rate is scaled as $1/n$. Max-Min's time grows faster (from 0.009 to 6.1 ms, about 670-fold), so the cost ratio narrows from about 5 250 to 128 as the workload grows.

XII. Conclusion

We reproduced the joint makespan and utilization optimization problem of [1] in CloudSim 5.0 and compared a GA scheduler with Round Robin, MCT, Min-Min and Max-Min on 50 to 1000 tasks with paired runs. The GA cut makespan by 54.5–57.5% and raised utilization from about 60% to 99.5% relative to Round Robin, and it was significantly better than MCT up to 200 tasks and than Min-Min up to 500 tasks. It did not beat Max-Min: they tie up to 200 tasks and Max-Min is 0.16–0.21% better at 500 and 1000 tasks while being 128 to 5 250 times cheaper to compute. In this setting the two objectives are aligned rather than competing; the GA's large-workload shortfall is caused by its mutation rate and disappears with $p_m = 1/n$; and the paper's own GA, reproduced exactly, is about 1% behind Max-Min up to 200 tasks and 9–29% behind at 500–1000 tasks because its mutation rate and iteration budget do not scale with the number of tasks. The closed-form model matched CloudSim to within 1.37% in every run. The GA's main benefit here is therefore flexibility, not raw performance.

XIII. Future Scope

- Use $p_m = 1/n$ and a convergence-based stopping rule instead of a fixed generation count.
- Seed the population with Max-Min and add a local-search step (e.g. swapping tasks between the most and least loaded VMs), which can improve on a seed that plain crossover and mutation cannot.
- Add objectives that do conflict with makespan, such as energy, monetary cost or SLA deadlines, where a GA's flexibility pays off, and use a Pareto method (e.g. NSGA-II).
- Compare with the metaheuristics used in [1] (PSO, ABC, BWO) and evaluate on real workload traces and larger, more heterogeneous VM pools.
- Extend to dependent-task workflows and to multi-PE VMs with a matching formulation.

References

[1] J. P. Dehury, D. Kumar, and S. Kumar, "Joint optimization of resource utilization and makespan in cloud computing using genetic algorithm," *Discover Computing*, vol. 29, art. no. 315, 2026, doi: 10.1007/s10791-026-10167-0.

[2] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011.

[3] A. Panda, "Cloud Computing Lab, Lab 1 (Software installation and running examples) and Lab 3 (Genetic algorithms)," lecture slides, Dept. of CSE, Birla Institute of Technology, Mesra, Ranchi, India.

[4] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: University of Michigan Press, 1975.

[5] O. H. Ibarra and C. E. Kim, "Heuristic algorithms for scheduling independent tasks on nonidentical processors," *Journal of the ACM*, vol. 24, no. 2, pp. 280–289, 1977.

[6] T. D. Braun *et al.*, "A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems," *Journal of Parallel and Distributed Computing*, vol. 61, no. 6, pp. 810–837, 2001.

[7] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[8] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 51–56.

[9] K. A. De Jong, "An analysis of the behavior of a class of genetic adaptive systems," Ph.D. dissertation, University of Michigan, Ann Arbor, MI, USA, 1975.

[10] J. J. Grefenstette, "Optimization of control parameters for genetic algorithms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 122–128, 1986.

[11] D. E. Goldberg and K. Deb, "A comparative analysis of selection schemes used in genetic algorithms," in *Foundations of Genetic Algorithms*, G. J. E. Rawlins, Ed. San Mateo, CA, USA: Morgan Kaufmann, 1991, pp. 69–93.

[12] T. Bäck, "Optimal mutation rates in genetic search," in *Proc. 5th Int. Conf. on Genetic Algorithms*, 1993, pp. 2–8.

[13] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[14] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.